BOXSPLOIT LABS / PRODUCT DOCUMENTATION

PANBOX

Product & operator guide

Self-hosted AppSec control plane. Setup, CI connectivity, access control, operations, and honest pilot limits.

VERSION 0.1 · 17 OCTOBER 2026 · PILOT

© 2026 BOXSPLOIT LABS

Contents

1. What runs where

2. Choose your setup path

3. Start the dashboard on Debian/Linux/macOS

4. Start the dashboard on Windows

5. Add only the agent to CI

6. Reach private PANBOX from public SCM

7. Give people only needed access

8. Create API, access, and scan keys

9. Confirm and control scans in the dashboard

10. Operate within the pilot boundary

11. Troubleshoot by failed step

12. Know the alpha limits

This guide describes implemented pilot behavior. Roadmap items are labelled separately.

© 2026 BOXSPLOIT LABS

Start with the path that matches your job: server operator installs the private dashboard once; DevOps adds an agent-only step to each CI job. The agent is one Go binary. The runner does not need Node, Python, Semgrep, Grype, or a PANBOX server installation.

1. What runs where

```
Git host/CI -> protected runner -> PANBOX agent -> private PANBOX API -> PostgreSQL
                                  | VACCIN: source rules            |
                                  | ENETRON: packages -> OSV        +-> React UI
```

Result: CI receives pass, gate failure, incomplete, or authorization failure; operators see repository and branch snapshots. Source files stay on the runner. ENETRON sends package names and versions to an OSV-compatible advisory API. The agent uploads normalized findings and a repository-bound scan key to PANBOX, without raw source snippets. See the architecture and network diagram.

2. Choose your setup path

- Server operator, once: Docker Engine/Desktop with Linux containers and Compose v2. Start with section 3 on Debian/Linux/macOS or section 4 on Windows. Debian is the initial production target.
- DevOps, per repository: enroll a repository, create one scan key, set CI secrets, and run section 5. The runner needs outbound HTTPS to GitHub Releases and the advisory API, plus a route to the private PANBOX API.
- No public IP/domain: choose a route in section 6 before writing the CI job.

PANBOX binds HTTP 8080 and PostgreSQL 5434 to host loopback by default. Change PANBOX_HOST_PORT or PANBOX_DB_HOST_PORT in .env if needed. Keep PostgreSQL private.

3. Start the dashboard on Debian/Linux/macOS

Goal: bring up the private API and UI once. Run on the server host:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
sh scripts/init-local.sh
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
curl -fsS http://127.0.0.1:8080/readyz
```

Expected: /readyz succeeds. Open http://127.0.0.1:8080 on the server and sign in with PANBOX_BOOTSTRAP_EMAIL and PANBOX_BOOTSTRAP_PASSWORD from the ignored .env. The initializer creates secrets with restricted permissions and refuses to overwrite an existing .env. Bootstrap credentials apply only when the database is first initialized; retain its volume on routine restarts.

Optional demo: PANBOX_SERVER=http://127.0.0.1:8080 bash scripts/seed-demo.sh needs curl and jq, then prompts for the admin password. Demo findings are synthetic.

4. Start the dashboard on Windows

Goal: run the same Linux containers through Docker Desktop. In PowerShell:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
./scripts/init-local.ps1
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
Invoke-RestMethod http://127.0.0.1:8080/readyz
```

Expected: the same dashboard and API as section 3. Use ./scripts/seed-demo.ps1 only for optional demo data. Run one initializer for a new .env, not both.

© 2026 BOXSPLOIT LABS

5. Add only the agent to CI

Onboarding model: PANBOX uses a register-first flow, similar to a SonarQube project key. The first scan does not create a repository or scan key automatically. This establishes the upload target and permissions before CI sends data. Enrollment only creates a PANBOX record; the form does not clone the source or require SCM credentials.

Once per repository, before changing the pipeline:

1. Sign in as an admin or a user with repos:write, then open Repositories -> Enroll repository. Select GitHub, GitLab, Bitbucket, Azure Repos, Gitea, or Other Git.

2. Enter name (display label, for example PANBOX), SCM repository ID or path (stable provider identity, for example alterwriter/PANBOX), default branch (for example main), and optional owner. A GitLab path such as group/subgroup/project works. Display name, SCM path, and internal PANBOX ID are three different values.

3. Select Enroll repository. The result dialog shows PANBOX ID for CI; copy this ID into CI variable PANBOX_REPOSITORY_ID. It remains available in the Repositories table under PANBOX ID for CI. Do not put the SCM path in this variable.

4. Select Generate scan key in the result dialog or repository row. The admin/key manager needs tokens:manage, scans:write, and repository access. Choose a label and expiry, then copy the one-time pb_scan_... secret into protected CI secret PANBOX_TOKEN. This key is bound to that PANBOX repository ID.

5. Open the row's Scan policy: enable scans, select VACCIN/ENETRON, and set the gate threshold. Set PANBOX_SERVER in CI to the API URL reachable from the runner, such as PANBOX's LAN address. 127.0.0.1 works only when runner and server share a host. Never put a scan key in repository files or untrusted pull request jobs.

6. From the runner, check connectivity, then check the key and ID with the commands below. When both work, run the agent job below. The first complete scan appears in Scans, with its findings in Findings.

```
curl -fsS "$PANBOX_SERVER/readyz"
curl -fsS -H "Authorization: Bearer $PANBOX_TOKEN" \
  "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config"
```

Every Linux/macOS job: download the pinned installer from PANBOX release v0.1.0-alpha.5. It selects OS/CPU, downloads just the binary, checks SHA256SUMS, and installs in the job workspace without sudo:

```
PANBOX_VERSION=v0.1.0-alpha.5
curl -fsSL
"https://github.com/alterwriter/PANBOX/releases/download/${PANBOX_VERSION}/install-agent.sh" -o
install-panbox-agent.sh
sh install-panbox-agent.sh "$PANBOX_VERSION" .panbox/bin
.panbox/bin/panbox go --server "$PANBOX_SERVER" --repo "$PANBOX_REPOSITORY_ID" --path .
```

Expected: exit 0 complete/pass; 1 gate failure; 2 incomplete/transport/config error; 3 rejected auth/permission. Copy the GitHub Actions job or GitLab CI job. On Windows, download install-agent.ps1 and run ./install-agent.ps1 -Version v0.1.0-alpha.5; it installs ./.panbox/bin/panbox.exe after checksum verification. Direct agent-only binaries are also available.

VACCIN checks selected Python, JavaScript/TypeScript, and Go source patterns. ENETRON matches pinned npm lockfiles v2/v3, requirements.txt, and go.mod dependencies against OSV advisory data. OSV supplies data, not scanner execution. PANBOX_ADVISORY_URL or --advisory-url selects an approved OSV-compatible mirror. --offline suppresses dashboard upload, but ENETRON still contacts the advisory API unless only --modules vaccin is selected.

CI-to-dashboard sequence: (1) the runner downloads the pinned agent; (2) panbox go authenticates GET /v1/repos/{id}/scan-config with the repository scan key; (3) the agent scans enabled modules in the checkout; (4) it posts one normalized JSON bundle to POST /v1/scans with Authorization: Bearer $PANBOX_TOKEN; (5) the API stores the snapshot and returns the gate verdict; (6) the agent exits 0 (pass), 1 (gate fail), 2 (incomplete/transport), or 3 (auth). There is no separate upload setup. Verify the policy from the runner with curl -fsS -H "Authorization: Bearer $PANBOX_TOKEN" "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config".

© 2026 BOXSPLOIT LABS

6. Reach private PANBOX from public SCM

- Local protected runner (recommended): polls any SCM over outbound HTTPS and posts to PANBOX on its LAN. No public PANBOX port; synchronous gate. Use the GitHub or GitLab example.
- SSH relay: PANBOX opens an outbound reverse tunnel to a reachable relay; a shared runner opens a forward tunnel. No PANBOX inbound internet port; synchronous gate. Pin host keys and restrict identities. See the GitLab relay example.
- Artifact import: CI writes panbox-results.json; a private host fetches and imports it later. No inbound route or relay; the later gate cannot block that CI job. GitLab artifact example and panbox relay-pull automate GitLab; other SCMs need their artifact download step before generic panbox go --bundle import.

Check: from a synchronous runner, curl -fsS "$PANBOX_SERVER/readyz" must work. A public repository does not make its CI artifacts public.

7. Give people only needed access

Admin setup: Settings & Audit -> RBAC -> create a role with explicit permissions -> assign users -> grant repository membership. admin is the highest immutable role. Built-in roles include security_reviewer, developer, executive_viewer, and scanner_ci; custom roles cannot receive rbac:manage.

Check: data access needs both permission and repository membership. Pilot permissions are repos:read, repos:write, scans:read, scans:write, findings:read, findings:recast, reports:export, rbac:manage, tokens:manage, and audit:read.

8. Create API, access, and scan keys

Where: Settings & Audit -> API & access keys. Choose label, type, expiry, scopes, and repository when applicable. API keys carry selected integration scopes; access keys are read-only; scan keys have scans:write for one repository. Copy the one-time secret into a secret manager. Keys are hashed at rest, expiring, and revocable.

Read-only smoke test:

```
curl -H "Authorization: Bearer $PANBOX_ACCESS_KEY" "$PANBOX_SERVER/v1/repos"
```

Browser login uses password, HttpOnly session cookie, and CSRF token. The OpenAPI contract lists pilot endpoints. Rotate by creating a new key, updating the client, confirming access, then revoking the old key.

9. Confirm and control scans in the dashboard

Recast: open Findings, clear filters if no rows appear, then select Recast severity beside a finding rule (the first column). Choose an effective severity or restore the scanner value -> enter a reason (10-500 characters) -> Save. The button appears only with findings:recast and repository access; an admin or security_reviewer with repository membership can use it. If no rows appear, complete the first scan or check filters. If rows appear without the button, ask an admin to check role and membership under Settings & Audit -> RBAC. Raw severity never changes. The API exposes history at GET /v1/findings/{id}/recasts; affected historic gate revisions are recalculated against each scan's original threshold. Recast is not an approval or dismissal.

Export: Reports -> choose repository, exact branch, scan module (VACCIN/ENETRON), and optional severity -> Export PDF, CSV, or JSON. Data comes from the latest complete scan per branch. PDF is a readable technical summary; CSV/JSON retain full rows. Requires reports:export.

MCP: Settings & Audit -> API & access keys -> create a read-only Access key with repos:read, scans:read, and findings:read. Open Settings & Audit -> MCP setup and use its generic stdio host configuration. The MCP host launches panbox mcp with PANBOX_SERVER and PANBOX_TOKEN in its secret environment. Four read-only tools list repositories, findings, scans, and scan detail. There is no public MCP endpoint.

© 2026 BOXSPLOIT LABS

After CI: Overview shows coverage and severity; Repositories shows enrolled projects; Scans shows run/gate status; Findings filters by repository/branch/module/severity; Reports exports filtered technical PDF/CSV/JSON; Settings & Audit shows RBAC, keys, and audit events. Light and dark themes are available.

Interpretation: a partial scan is unverified and preserves the last successful posture. ARK labels Critical/High and BUG-STER labels Medium/Low while standard severities remain stored and exported. Recast actions and changes to scan policy are audited.

10. Operate within the pilot boundary

Keep PostgreSQL on loopback. For remote browser access, use an HTTPS reverse proxy with access control or a private route; set PANBOX_SECURE_COOKIE=true for HTTPS. Back up PostgreSQL and test restoration.

The 2 GB Debian host is a target to validate, not a guarantee. A small Windows Docker Desktop fixture measured about 5.1 MiB server RAM and 37.3 MiB PostgreSQL RAM; 100 sequential API calls measured 7.1 ms median and 9.3 ms p95. The agent runs on the runner and needs separate sizing. Measure docker stats --no-stream with representative repositories and scan bundles.

11. Troubleshoot by failed step

- Dashboard not ready: run docker compose --env-file .env -f deploy/docker-compose.yml ps and logs db server; confirm PostgreSQL health.
- Agent download fails: check release tag, OS/CPU asset name, outbound GitHub access, curl, and SHA-256 tooling. Do not skip the checksum.
- Runner cannot reach PANBOX: 127.0.0.1 inside a container is that container; use its reachable private host/network address or the SSH relay.
- Exit 2: check manifest, advisory endpoint, work path, PANBOX endpoint, and logs; incomplete is not pass.
- Exit 3: check scan key expiry/revocation, scans:write, internal repository ID, role, and membership.
- Artifact import is late: check timer, GITLAB_READ_TOKEN, allowed refs, Jobs API/artifact access, and relay state file.

12. Know the alpha limits

Available now: server/UI, native VACCIN source checks, native ENETRON pinned dependency matching with OSV data, repository/branch snapshots, gate, RBAC, scoped keys, audit, filtered PDF/CSV/JSON exports, and the transport examples above. Validate real runners, OSV access or approved mirror, Debian load, backup/restore, TLS, and key rotation in your environment.

Coverage is intentionally narrow. RIDER secrets, TESLA IaC, approval workflows, executive PDF templates, AI Security, OIDC, and API rate limiting are roadmap items. This PDF is a product guide, not a PANBOX vulnerability report.

© 2026 BOXSPLOIT LABS